

Reviewer Pack — Sample-Compression Gap Predicts Worst-Case DPLL Depth on Ran...

Entry 318f60b6c3b3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-26 23:56:31 UTC

Sample-Compression Gap Predicts Worst-Case DPLL Depth on Random 3-CNF

Entry ID: 318f60b6c3b3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-26 23:56:31 UTC

1. Conjecture

Field A (mathematical branch): PAC-Bayesian sample compression schemes (Littlestone-Warmuth compression sets) **Field B** (complexity object): Worst-case DPLL search-tree depth on random 3-CNF formulas

Statement:

For random 3-CNF instances at clause density $\alpha=4.0$ over n variables, let $C(F)$ be the size of the smallest ‘partial-assignment compression set’: the minimum k such that there exist k literals whose unit-propagation closure under F either derives a contradiction or fixes a satisfying assignment. We conjecture that the worst-case DPLL depth $D(F)$ (with the JW-OS heuristic) satisfies $C(F) \leq D(F) \leq 3 \cdot C(F) + O(\log n)$ for at least 90% of unsatisfiable instances at $n \leq 20$, i.e. the average-case compression invariant pins worst-case search depth up to a constant factor.

Rationale (proposer’s reasoning):

Sample-compression theory (Littlestone-Warmuth, Moran-Yehudayoff) translates average-case learnability into combinatorial certificates of bounded size; mapping this to SAT, a small compression set is precisely a short ‘witness path’ that any reasonable DPLL must re-discover. The bridge is rare because compression schemes are normally posed for hypothesis classes, not CNF instances, yet both measure the minimal prefix forcing a decision. If true, this gives a constructive average-to-worst-case reduction: average-case bounds on $C(F)$ under the planted-3CNF distribution would lift directly to worst-case DPLL depth lower bounds.

Taxonomy category: AVG_TO_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 45556438eae6c2f6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 350 random unsat 3-CNF instances ($n \in \{8, 10, 12, 14, 16, 18, 20\}$, 50 seeds each, $\alpha=4.0$), per-instance check $C(F) \leq D_{\text{JW-OS}}(F) \leq 3 \cdot C(F) + \lceil \log_2 n \rceil$. Conjecture SUPPORTED iff support fraction ≥ 0.90 overall AND ≥ 0.80 at every n . FALSIFIED if either threshold missed, or any instance has $D(F) > 3 \cdot C(F) + \lceil \log_2 n \rceil + 2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - sample compression DPLL proof complexity random 3-CNF - Littlestone-Warmuth compression scheme resolution width unit propagation - PAC-Bayes compression set SAT solver search tree depth random k-SAT

Top relevant hits considered: - [<http://arxiv.org/abs/1503.01613v3>] Space proof complexity for random 3-CNFs - [<http://arxiv.org/abs/cs/0506069v1>] A generating function method for the average-case analysis of DPLL - [<http://arxiv.org/abs/cs/0209032v3>] Complexity Results on DPLL and Resolution - [<http://arxiv.org/abs/1810.01864v2>] Agnostic Sample Compression Schemes for Regression - [<http://arxiv.org/abs/2401.04670v1>] Modified Levenberg-Marquardt Algorithm For Tensor CP Decomposition in Image Compression - [<http://arxiv.org/abs/2310.10886v1>] The Calysto Scheme Project - [<http://arxiv.org/abs/1002.1290v3>] Bounds on Threshold of Regular Random k -SAT - [<http://arxiv.org/abs/1402.4455v1>] Symbiosis of Search and Heuristics for Random 3-SAT

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.6s

5.1 Generated Python source

```
import random
import math
import json

def gaussian_elimination(A, b):
    n = len(b)
    A_b = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A_b[j][i]) > abs(A_b[max_row][i]):
                max_row = j
        A_b[i], A_b[max_row] = A_b[max_row], A_b[i]
        factor = A_b[i][i]
        for j in range(n + 1):
            A_b[i][j] /= factor
```

```

        for j in range(n):
            if i != j:
                factor = A_b[j][i]
                for k in range(n + 1):
                    A_b[j][k] -= factor * A_b[i][k]
    return [row[-1] for row in A_b]

def unit_propagation(clauses, assignment):
    while True:
        new_assignment = False
        for clause in clauses:
            if all(lit not in assignment or (lit < 0) == (assignment[lit] is False) for lit in clause):
                continue
            unassigned_lits = [lit for lit in clause if lit not in assignment]
            if len(unassigned_lits) == 1:
                new_assignment = True
                assignment[unassigned_lits[0]] = (unassigned_lits[0] > 0)
        if not new_assignment:
            break
    return assignment

def dpll(clauses, assignment):
    n = max(abs(lit) for clause in clauses for lit in clause)
    if all(lit in assignment or (lit < 0) == (assignment[-lit] is False) for clause in clauses):
        return len(assignment)
    unassigned_vars = [var for var in range(-n, 0) if var not in assignment]
    var = unassigned_vars[0]
    assignment[var] = True
    result = dpll(clauses, assignment)
    if result is not None:
        return result
    assignment[var] = False
    return dpll(clauses, assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([8, 10, 12, 14, 16, 18, 20])
    alpha = 4.0
    m = int(alpha * n * (n - 1) / 6)
    clauses = []
    for _ in range(m):
        while True:
            lits = random.sample(range(-n, 0), 3)
            if len(set(lits)) == 3 and all(abs(lit) != abs(lit2) for lit, lit2 in zip(lits, lits[1:])):
                clauses.append(lits)
            break
    assignment = {}
    C_F = None
    for k in range(1, n + 1):
        for subset in itertools.combinations(range(-n, 0), k):
            sub_clauses = [clause for clause in clauses if any(lit in subset for lit in clause)]
            if unit_propagation(sub_clauses, assignment) is None:
                C_F = k
                break
    if C_F is not None:
        break

```

```

D_F = dpll(clauses, {})

if D_F is None or D_F > 3 * C_F + math.ceil(math.log2(n)) + 2:
    return {
        "metric_name": "DPLL Depth",
        "metric_value": D_F,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"D(F) > 3*C(F) + log2(n) + 2"
    }

    return {
        "metric_name": "DPLL Depth",
        "metric_value": D_F,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.90 and all(res["support_fraction"] >= 0.80 for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(res["conjecture_holds"] is False for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"D(F) > 3*C(F) + log2(n) + 2\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} < 0.90")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2558e90.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2558e90.py", line 87, in run_trial
    D_F = dpll(clauses, {})
         ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2558e90.py", line 53, in dpll

```

```

if all(lit in assignment or (lit < 0) == (assignment[-lit] is False) for clause in clauses):
~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2558e90.py", line 53, in <genexpr>
    if all(lit in assignment or (lit < 0) == (assignment[-lit] is False) for clause in clauses):
~~~~~

```

```
NameError: name 'lit' is not defined. Did you mean: 'List'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `NameError` (`'lit'` is not defined) inside the DPLL satisfaction check before any trial completed, so no data was produced to evaluate the pre-registered criterion. | next: Fix the buggy generator expression in `dpll`'s clause-satisfaction check (iterate over clauses then literals, e.g. `all(any(... for lit in clause) for clause in clauses)`) and rerun the 350-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	22042
2	preregistration	claude_max	opus	0	0	6652
3	novelty	claude_max	opus	0	0	3346
4	novelty	claude_max	opus	0	0	8773
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10729
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13661
7	judge	claude_max	opus	0	0	4663

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 69866 ms total latency. Provider mix: {'claude max': 5, 'ollama remote': 2}

(full prompt+response transcripts available in [research/audit/318f60b6c3b3.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/318f60b6c3b3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp sandbox/test *.py (per-cycle)

Replay tarball: `research/replay/318f60b6c3b3.tar.gz` (if generated)